KABC Benchmark: Results (Experiments 1–4, Current Redesign)

Kernel-based Active Clustering of Bandit Arms
Thuot, Vogt, Ghoshdastidar, Verzelen (arXiv:2601.07535)

September 17, 2026

This report summarizes the experimental setup, baseline algorithms, and results for Experiments 1–4 of the `kernel-active-bandit` benchmark suite. Experiments 1–3 stand as they were after a substantial redesign of all three (algorithm roster, grids, and figures all changed from the previous report). Full design rationale lives in `docs/experiments.md`; this document summarizes what was actually run and what the results show, plus a full pass at inferences. **All four experiments are now complete at their full planned 50 reps. Experiment 4 was substantially redesigned on 2026-09-16** (Section 1.4) and its full 50-rep production run under the new design finished cleanly on 2026-09-17 — see Section 8 for the real results, and the Run Provenance paragraph below.

Run provenance. `results/exp1/results.parquet` holds 6,300 rows (14 grid points × 9 algorithms × 50 reps) — **complete at the planned 50 reps**. 13 of these 14 points are the canonical grid; the 14th ($N = 90$) fills the gap between N=60 and N=120 on the N sweep, added permanently to `configs/exp1.yaml`. `acb-gaussian` (Section 2.3) was added as a ninth algorithm row on 2026-09-15, after the other eight's 50-rep run had already completed — merged in via a separate scratch-run-and-merge (REVISION 6, `docs/experiments.md`), not a resume of the original run. `results/exp2/results.parquet` holds 2,400 rows (6 settings × 8 algorithms × 50 reps) — **complete at the planned 50 reps**. `results/exp3_{1,2,3}/results.parquet` hold 300 rows each (6 algorithms × 50 reps) — **complete at the planned 50 reps**. All three configs share the same underlying architecture: dedicated scripts (`scripts/run_experiment1/2/3.py`), calibrated threshold constants (Section 3), and figures regenerated by `notebooks/01_gaussian_mixtures.ipynb`, `notebooks/02_same_mean_shapes.ipynb`, and `notebooks/03_movielens_random_pairs.ipynb` respectively. All three experiments' `-0.9x` self-comparison rows were renamed to `-0.5x` this round (Section 2 explains why). Experiment 4's redesigned `results/exp4_baseline/results.parquet` and `results/exp4_amplified_imbalance/results.parquet` each hold 100 rows (2 algorithms × 50 reps) — **complete at the planned 50 reps**, produced by the new `scripts/run_experiment4.py`; figures regenerated by `notebooks/04_movielens_prototypes.ipynb`. See Section 8 for the real results.

Contents

1 Experimental Setup **2**

1.1 Experiment 1 — Gaussian mixtures: competitiveness in the parametric regime . . . 2

1.2 Experiment 2 — Identical means, different shapes: nonparametric separation 3

1.3 Experiment 3 — MovieLens random pairs: robustness across the real-data pair distribution 4

1.4 Experiment 4 — MovieLens prototypes: kabc's decision rule vs. unif-kkm's k-means under real imbalance 4

2 Baseline Algorithms **5**
2.1 KABC and KABC-SG (this paper's own methods) 5
2.2 ACB — Active Clustering with Bandit feedback 6
2.3 ACB-Gaussian — the paper's own Gaussian-specialized, non-adaptive variant 6
2.4 TaS-FW — Track-and-Stop with Frank-Wolfe 7
2.5 Unif-KKM — uniform sampling + kernel k-means (folklore baseline) 7
2.6 Related active-clustering methods not evaluated 8

3 Calibration **8**
3.1 Implementation robustness: two properties worth stating explicitly 9

4 Results — Experiment 1 (50 reps, 14 grid points, complete) **10**

5 Results — Experiment 2 (50 reps, 6 settings, complete) **13**

6 Results — Experiment 3 (50 reps, 3 versions, complete) **16**

7 Results — Experiment 3 follow-up: tas-fw low-budget probe and calibration **20**

8 Results — Experiment 4 (clean redesign, complete at 50 reps) **22**

9 Inferences and Conclusions **24**

10 Appendix: why Experiment 4's `budget_cap` increase and its memory-guard override are two independent changes **27**

11 Appendix: how loose is Experiment 1's `budget_cap` relative to the paper's own Theorem 3.1 bound **27**

1 Experimental Setup

1.1 Experiment 1 — Gaussian mixtures: competitiveness in the parametric regime

Claim tested. In the mean-separated Gaussian setting (home turf of mean-based methods like ACB), KABC's budget is within a modest factor of ACB's; budget scales as $s_*^{-2}\log(N/\delta)$ (the paper's Theorem 3.1). Also (current redesign): how conservative each adaptive method's own stopping rule is, and whether active/adaptive sampling beats naive uniform sampling at a matched budget.

Instance family. `GaussianMixtureEnv`: N arms in $K = 3$ balanced clusters, arm i in cluster k distributed as $\mathcal{N}(\mu_k, \sigma^2 I_d)$, $\sigma = 1$. Cluster means $\mu_k = \frac{\Delta}{\sqrt{2}}(e_{2k-1} + e_{2k})$, giving each pair of cluster means an actual Euclidean separation of exactly Δ.

Grid. One knob swept at a time, others held at the base config N=60, K=3, d=10, Δ=3:

- $\Delta \in \{1, 1.5, 2, 3, 4, 6\}$ (6 points — the headline sweep)
- $d \in \{6, 10, 50, 100\}$ (4 points; $d = 2$ excluded, construction requires $d \geq 2K = 6$)
- $N \in \{30, 60, 90, 120\}$ (4 points; $N = 90$ added this round to fill the gap between 60 and 120)

14 grid points total (the base config is each sweep's own midpoint, so it is run identically under all three sweep tags — a documented, accepted redundancy).

Algorithms: 8 rows per (grid point, rep). Three independent BASE algorithms — `kabc`, `kabc-sg`, `acb` — each run once at the grid point's full `budget_cap`. Each base is then, separately, re-run at budget_cap = $\min(0.5 \times \tau_{\text{own}}, \text{budget_cap})$ with a fresh independent RNG substream — `kabc-0.5x`, `kabc-sg-0.5x`, `acb-0.5x` respectively, each testing how conservative THAT algorithm's own stopping rule is. `unif-kkm-1x` and `unif-kkm-0.5x` are tied specifically to `kabc`'s own τ (not kabc-sg's or acb's), testing whether active/adaptive selection beats naive uniform sampling at a matched budget. No `tas-fw` here: its finite-alphabet assumption does not hold for continuous Gaussian instances (Experiment 3 only).

Budget_cap. One value per grid point, shared across all 8 algorithm rows and every rep: budget_cap = $200\, N\, s_*^{-2} \log(N/\delta)$ (Theorem 3.1), s_*^2 Monte-Carlo-estimated (`kabc.metrics.estimate_s_star_sq`).

1.2 Experiment 2 — Identical means, different shapes: nonparametric separation

Claim tested. When clusters share the same mean, mean-based methods are blind (ARI at chance for any budget) while KABC recovers the partition with budget governed by the MMD gap. Also (current redesign): the same stopping-rule-conservativeness and active-vs-uniform-sampling questions as Experiment 1.

Instance families and the current 6-setting grid. Three families, $N = 40$, $K = 2$ throughout. As of this report, the grid is **6 fixed, illustrative settings** (not a sweep whose in-between variation is the point — see `docs/experiments.md`'s Experiment 2 GRID TRIM NOTE for the rationale):

- `scale_pair` (2a): cluster A = $\mathcal{N}(0, 1)$, cluster B = $\mathcal{N}(0, \sigma_B^2)$ — same mean, different *variance*. $\sigma_B \in \{2, 3\}$.
- `moment_matched_mixture` (2b): cluster A = $\mathcal{N}(0, 1)$, cluster B = $0.5\,\mathcal{N}(-a, b^2) + 0.5\,\mathcal{N}(a, b^2)$, $b^2 = 1 - a^2$ — same mean *and* variance, different *shape*. $a \in \{0.85, 0.9\}$.
- `ring_blob` (2c): cluster A = a 2D ring (Gaussian noise $\sigma = 0.1$), cluster B = $\mathcal{N}(0, I_2)$, both centered at the origin — same mean, different *2D geometry*. Ring radius $r \in \{2, 3\}$.

50 repetitions per setting (full planned run, see Run Provenance above).

Algorithms: 8 rows per (setting, rep), identical scheme to Experiment 1. Same base/0.5x-self-comparison/unif-kkm-tied-to-kabc structure as Experiment 1 (see above). No `kabc-oracle` (dropped — its one-shot required budget exceeded `budget_cap` at every grid point of the old design, so the "price of adaptivity" figure it existed for never produced real data). No `tas-fw`, same reason as Experiment 1.

Budget_cap. Identical mechanism to Experiment 1: one s_*-formula value per setting, shared across all 8 rows and every rep.

1.3 Experiment 3 — MovieLens random pairs: robustness across the real-data pair distribution

Claim tested. Whether KABC's advantage (and ACB's/TaS-FW's relative behavior) holds up when the movie PAIR is randomly resampled every repetition across the real diversity of the MovieLens catalog, under three distinct pairing regimes that vary how much mean separation is present and whether the second movie is cherry-picked (max MMD^2) or sampled uniformly. Also: how conservative KABC's own stopping rule is.

Data. `data/ml-1m-207/` — the 207 MovieLens-1M movies with ≥ 1000 ratings. Discrete rating support $\{1, \ldots, 5\}$, RBF-on-ratings kernel, fixed median-heuristic bandwidth ($= 1.0$) computed once over the full 207-movie pool.

Instance. $N = 8$ arms, $K = 2$ clusters, 4 arms per prototype (movie), `MovieLensPairEnv`. For a given rep, a single shared permutation of all 207 movie indices is drawn; each of the three versions independently scans that same permutation and picks the first candidate movie A with at least one eligible B under *its own* gap rule (so versions usually, but not always, land on the same A — see `docs/experiments.md` for the precise mechanism).

Version-specific B-selection rules.

`exp3_1` — **tight mean gap, MMD-maximizing.** Candidates: all movies with $|\text{mean}_A - \text{mean}_B| \leq 0.25$. $B = \arg\max \text{MMD}^2$ among candidates.

`exp3_2` — **moderate-to-large mean gap, uniform.** Candidates: all movies with $0.25 \leq |\text{mean}_A - \text{mean}_B| \leq 1.0$. B sampled uniformly at random among candidates.

`exp3_3` — **tight mean gap, uniform.** Candidates: all movies with $|\text{mean}_A - \text{mean}_B| \leq 0.25$ (same tolerance as `exp3_1`, a clean matched-tolerance ablation of cherry-picking vs. uniform B-selection). B sampled uniformly at random.

Budget. Fixed budget_cap = 15,000 pulls (not the s_*-formula type — not meaningful at this instance scale). Deliberately not raised to clear `kabc`'s doubling-trick round-7 threshold (28,560 at $N = 8, \delta = 0.05$): this leaves `kabc`'s own non-convergent behavior unchanged while giving `acb`/`tas-fw` less runway.

Algorithms: 6 rows per (rep, version). `kabc`, `kabc-0.5x`, `acb`, `tas-fw`, `unif-kkm-1x`, `unif-kkm-0.5x` — the same base/0.5x/unif-kkm-tied-to-kabc scheme as Experiments 1–2, restricted to the three algorithms for which it is meaningful here (`kabc-sg` dropped this round — its fragility on this exact real-data setting is already thoroughly documented from a prior run, see `docs/experiments.md`'s REVISION 4 note). 50 repetitions per version (full planned run).

1.4 Experiment 4 — MovieLens prototypes: kabc's decision rule vs. unif-kkm's k-means under real imbalance

Clean redesign (2026-09-16), superseding an earlier 4-condition/6-algorithm version entirely. That version's own results are not reported here; see `docs/experiments.md`'s Experiment 4 section (git history) for what it found and exactly why each change below was made.

Claim tested. Is kabc's decision rule (pairwise variance-aware threshold + connected components) more robust than unif-kkm's k-means under heterogeneous difficulty and real cluster-size imbalance? Both methods run at the SAME (matched) budget on real MovieLens-1M rating distributions as prototypes.

Two algorithms only. `kabc`; `unif-kkm-1x` (budget = that rep's own realized $\tau_{\texttt{kabc}}$, the matched-budget design) — the earlier version's $\tau_{\texttt{kabc}}$-blind fixed-budget rows served their purpose (establishing unif-kkm loses even with MORE budget than kabc used) but are not needed for this report.

Two settings, both real-data mean-gap-resampled every rep (no hand-picked fixed instance). Both mirror Experiment 3's own `exp3_3` mechanism (bounded mean-gap eligibility, uniform pick among eligible, not MMD^2 cherry-picking): a close pair (mean gap ≤ 0.25) and a far pair (mean gap ≥ 0.5) guaranteed each rep, plus unconstrained generic anchors. `baseline`: K=7, `far_large`=12, `far_small`=2, `close_a`=5, `close_b`=4, 3 generic anchors (8, 6, 3) — N=40. `amplified_imbalance`: K=12, `far_large`=20, `far_small`=2 (a 10:1 ratio, not `baseline`'s 6:1), same close pair sizes, 8 generic anchors (10, 8, 7, 6, 5, 4, 3, 2) — N=76; this is the regime where the earlier design's K=12 condition showed the cleanest imbalance-specific signal (unif-kkm's far-pair ARI below its close-pair ARI, small-prototype misclassification ∼1.6–2× large-prototype), pushed further.

Memory guard override, real-verified. Both settings run at n_{jobs}=5 (not Experiments 1–3's 10–11) with `kabc.fit`'s own additive, opt-in memory-guard override (`memory_guard_n_workers`=5, `memory_guard_safety_factor`=2 — Experiments 1–3 are entirely unaffected, keeping the original 11-worker/4× assumption). This raises the safe per-iteration ceiling from n_k∼2852 to n_k∼5983, verified with real measurements before use, not just the relaxed formula: a real k=8-scale computation (n≈4602, all pairs, N=40) measured 627.7 MB actual peak RSS per worker (vs. the pure 3-Gram-matrix model's 484.9 MB); 5 real concurrent OS processes at that scale measured 626.4–626.5 MB each (sum 3.06 GiB, wall-clock 1242.4s), with `vm_stat` confirming zero new swapouts during the run. This raises kabc's honest convergence ceiling from k≤7 to k≤8 for both settings (*budget_cap* raised correspondingly: 400,000 for `baseline`, 800,000 for `amplified_imbalance`, each covering k=8's full cumulative cost with margin); k=9 (n_k≈9200–9900) was checked and rejected — real projected memory at 5 workers is ∼14.5 GiB, too close to this machine's 16GB target. Full derivation in `docs/experiments.md`.

Metrics. Standard τ/`stopped`/ARI, plus a per-pair ARI breakdown (`ari_close_pair`, `ari_far_pair`) and a per-prototype-size misclassification rate (`misclass_rate_large`/`_small`, via best-overlap alignment, median-split per instance) — both carried over unchanged from the earlier design.

2 Baseline Algorithms

2.1 KABC and KABC-SG (this paper's own methods)

KABC (Algorithm 2 of [1]) runs a doubling-trick outer loop: at iteration k, pull $n_k = \lceil 2^k \log(8(N^2 - N)/\delta_k) \rceil$ fresh samples per arm ($\delta_k = \delta/(4k^2)$), then build a same-cluster graph over all arm pairs via a pairwise MMD-based test — an edge $\{i, j\}$ is added iff the empirical MMD statistic between arms i and j falls under a threshold. The recovered partition is the graph's connected components; the loop stops the first time the number of connected components equals the true K.

Two threshold variants are compared (identical outer loop, different pairwise test):

`kabc` variance-aware threshold (Eq. 6 of [1], building on the variance-aware kernel mean embedding estimates of [4]): sizes each pair's own threshold from that pair's *empirical* RKHS variance, so a noisier pair automatically gets more slack.

`kabc-sg` kernel-uniform sub-Gaussian threshold (the Discussion section's ablation in [1], substituting the minimax kernel-mean-embedding bound of [5], Proposition A.1): uses only the fixed kernel bound (RBF kernel sup = 1), identical for every pair regardless of that pair's own spread — the ablation whose practical fragility Section 9 discusses in detail.

Implementation verification. `src/kabc/thresholds.py`'s `variance_aware_threshold` and `subgaussian_thr` were checked directly against Eq. 6 and the Discussion section's substitute formula in [1] and match exactly, term for term (including the 32/3 coefficient in Eq. 6's second term and the $(\sqrt{\cdot} + 2)$ form of the sub-Gaussian bound) — including correctly distinguishing $\tilde{g}$ (the kernel's *range*, $\sup - \inf$, used in Eq. 6) from $\bar{g}$ (the kernel's *supremum*, used in the sub-Gaussian formula): these two quantities coincide numerically for the RBF kernel used throughout this repo ($\tilde{g} = \bar{g} = 1$, since the RBF kernel's infimum is 0 in the limit), but are conceptually distinct quantities in the paper, and the code keeps them as separate named parameters (`g_tilde` vs. `g_bar`) rather than silently conflating them.

The `-0.5x` self-comparison rows (current redesign, all three experiments). For each base algorithm (`kabc`, `kabc-sg`, `acb`), a second, independent run at budget_cap $= \min(0.5 \times \tau_{\text{own}}, \text{budget_cap})$ tests how much margin that algorithm's own stopping rule carries. **Why 0.5x, not 0.9x:** an earlier version of this design used a 0.9x multiplier; direct per-rep inspection of the resulting `kabc`/`kabc-0.9x` data showed KABC's doubling-trick schedule quantizes its own realized τ to a small set of round-boundary checkpoints, and the FINAL round alone typically costs roughly half of the total realized τ. A 0.9x budget cut therefore almost always landed short of that final round regardless of how close to 1.0 the multiplier nominally was — there was no way to construct a real "kabc re-run with 10% less budget" against this schedule; 0.9x and, say, 0.6x tested essentially the same thing. 0.5x makes what is actually being tested explicit: a meaningfully smaller, still-real budget cut. This replaces an earlier, cruder budget_multiplier $\in \{1.0, 0.25\}$ restricted-budget pass applied uniformly across all algorithms.

2.2 ACB — Active Clustering with Bandit feedback

Source: [2]. ACB* (this repo's adaptive variant, "θ_* and Δ_* unknown," referred to throughout as ACB) is a *mean*-based active clustering procedure (SRI + ADC subroutines): it estimates each arm's mean directly (no kernel/MMD machinery) and clusters by Euclidean distance between empirical means, adaptively refining which arms to pull based on estimated separation. It is

the "home turf" competitor for Experiment 1 (mean-separated Gaussians, exactly ACB's design setting) and the mean-blindness foil for Experiments 2–3 (same-mean, different-shape data, where a mean-only method has no signal by construction).

2.3 ACB-Gaussian — the paper's own Gaussian-specialized, non-adaptive variant

Source: [2], Appendix A. Beyond the main-text ACB* above, the paper's own numerical experiments (its Figure 1, reporting $\leq 1\%$ error at $\delta = 0.1$) do not actually use ACB*'s adaptive Algorithm 2 — they use a separate, non-adaptive variant (Algorithm 1, with Δ_* and θ_* assumed known), with every concentration-inequality-derived constant in Eq. 9–13 replaced by an exact δ-level quantile: n_{max} from a χ^2 quantile, and I/J from a normal quantile β and a Monte-Carlo-estimated product-quantile α. We implement this variant separately (`acb-gaussian`, `src/kabc/algorithms/acb_gaussian.py`) as a ninth, independent row in Experiment 1's algorithm roster only (no `-0.5x` self-comparison counterpart).

This baseline is evaluated with Δ and θ known, matching the non-adaptive Algorithm 1 setting the paper's own authors use for their Figure 1 experiment — not the fully adaptive ACB* — since this is the fairest, most literal reproduction of the comparison the paper's own authors ran. We state this explicitly here rather than leaving it implicit: a reader could otherwise reasonably wonder why one baseline gets privileged information (the true separation Δ and balancedness θ) that no other algorithm in this benchmark receives. The answer is that Experiment 1's `GaussianMixtureEnv` instances are *constructed from* these exact values (`configs/exp1.yaml` passes Δ and σ directly to the environment's constructor), so nothing is being estimated from data that a fair comparison would otherwise require estimating — Δ and θ are known at instance- construction time in the same sense N, K, and d already are to every algorithm in this repo. This does mean `acb-gaussian` does not match CLAUDE.md's standard `fit(env, delta, budget_cap, rng)` interface in substance (only in call shape: $\Delta/\theta/\sigma$ are read directly off `env`, not estimated), so it is wired only into Experiment 1's own algorithm registry and hard-fails immediately if ever invoked on a non-Gaussian environment, rather than silently misapplying a Gaussian-only method outside its valid domain.

No calibration. Unlike `kabc`/`kabc-sg`/`acb`, `acb-gaussian` has no calibrated constant in `configs/calibration.yaml` (the entry there is present and explicitly documented as a deliberate, permanent absence, not a not-yet-run placeholder): every quantity is an exact δ-level quantile of a known finite-sample distribution, not a concentration- inequality bound with slack to tighten empirically.

2.4 TaS-FW — Track-and-Stop with Frank-Wolfe

Source: [3]. Track-and-Stop based on Frank-Wolfe optimization, for N-ary clustering of arms whose distributions live on a common *finite* alphabet. Only run in Experiment 3, where MovieLens ratings on $\{1, \ldots, 5\}$ satisfy the finite-alphabet assumption natively; not run on Experiments 1–2's continuous instances, which would require an off-label discretization approximation outside the algorithm's proven guarantees. Its Track-and-Stop design re-scans its entire hypothesis space and re-solves a linear program at *every single pull* (not periodically, like KABC's doubling trick), which is why Experiment 3 caps N at 8: the hypothesis space grows combinatorially in N, and empirically `tas-fw` already costs two to three orders of magnitude more wall-clock per repetition than every other algorithm at $N = 8$ (confirmed again at the full 50 reps, Section 6).

2.5 Unif-KKM — uniform sampling + kernel k-means (folklore baseline)

No paper — a standard, non-adaptive baseline implemented directly per the design doc. Pulls a fixed number of samples per arm, then runs ordinary kernel k-means on the resulting empirical kernel mean embeddings. Has no confidence-based stopping rule and no δ-PAC guarantee — it always spends its entire allotted budget and always "completes" (stopped = True by construction), so it is included purely as a non-adaptive reference point, not as a δ-correct competitor. `unif-kkm` no longer sweeps an independent budget — its two rows, `unif-kkm-1x` and `unif-kkm-0.5x`, are tied specifically to `kabc`'s own realized τ for that same rep (budget_cap = $\min(1.0 \text{ or } 0.5 \times \tau_{\texttt{kabc}}, \text{budget_cap})$), directly testing whether active/adaptive selection beats naive uniform sampling at a matched budget.

2.6 Related active-clustering methods not evaluated

Several other recent papers address closely related active/bandit clustering problems but were not included as baselines, either because their problem setting differs from ours in a way that would make a head-to-head comparison uninformative or unfair, or because of prohibitive computational cost at our benchmark's scale.

Optimal Clustering with Bandit Feedback [6] proposes BOC, an asymptotically optimal Track-and-Stop-style algorithm for the same fixed-confidence clustering problem we study. We do not include it as a baseline because Track-and-Stop methods require solving a constrained optimization problem at every sampling round, making per-decision cost prohibitive at our problem sizes; we observed comparable cost issues with `tas-fw` [3], which we retain only on a small reduced instance (Experiment 3, $N = 8$) for this reason.

Clustering Items through Bandit Feedback: Finding the Right Feature out of Many [7] studies a distinct observation model in which the learner selects both an item and a specific feature at each round, rather than observing a full feature vector per pull. This partial-observation setting is not directly comparable to our full-vector-feedback setting without a nontrivial reformulation of the environment, and the method (BanditClustering) is designed specifically around exploiting the sparsity of the discriminative signal across features, which is not the structure our instances test.

Near Optimal Best Arm Identification for Clustered Bandits [8] addresses a fundamentally different goal: given N agents each facing their own K-armed bandit problem, with agents implicitly clustered by which bandit problem they share, the objective is to identify each agent's best arm, not to recover the clustering as an end in itself. Clustering is a means to accelerate best-arm identification via information-sharing across agents in the same cluster, whereas partition recovery is our sole objective; the two problems are not directly comparable on the same metric.

Online Clustering of Data Sequences with Bandit Information [9] proposes ATBOC for a setting close to ours (fixed-confidence clustering of M arms into K groups from bandit feedback), permitting non-identical within-cluster distributions and vector observations. We did not implement it as a baseline in the current benchmark; a natural extension of this work would include it as an additional parametric competitor alongside `acb`, since its algorithmic structure (average tracking) differs from both ACB's mean-based approach and BOC's Track-and-Stop approach.

Efficient Clustering in Stochastic Bandits [10] proposes EBC, which addresses the same per-round optimization bottleneck that makes BOC and `tas-fw` expensive, by taking a single gradient-like step toward the optimum at each round rather than solving the full optimization problem, while retaining asymptotic optimality. This is a promising, more efficient alternative to BOC that emerged concurrently with our own work; we did not have the opportunity to incorporate

it into the present benchmark, but it is a natural inclusion for future comparison given it targets the exact same computational bottleneck we cite as our reason for excluding BOC and limiting `tas-fw`'s scope.

3 Calibration

`kabc`, `kabc-sg`, and `acb` each have a theoretically-derived but conservative leading constant in their stopping threshold. Per the calibration protocol (`docs/experiments.md` §0.2), each constant was empirically calibrated once (binary search for the smallest constant keeping empirical error $\leq \delta/2$, validated under the REAL formula-derived budget regime, not an artificially generous one), then frozen and reused unchanged across every experiment. The frozen values actually used for this report:

Algorithm	c_{cal}	Calibration-set error	Held-out error
`kabc`	0.103516	0.000	0.000
`kabc-sg`	0.121094	0.000	0.000
`acb`	0.658984	0.020	0.010
`tas-fw`	— (no calibratable constant)	—	—

Important caveat. Calibration instances are a *single* fixed Gaussian-mixture instance (`GaussianMixtureEnv`'s construction is deterministic given N, K, d, Δ — there is no way to draw "20 different instances," only 20 independently-seeded noise draws of the one instance). The frozen constants above were validated against that one instance's own real budget regime, *not* against every real grid point in Experiments 1–3. Section 9 shows this still matters directly: `kabc-sg`'s calibration does not fully generalize to the harder grid points/settings in either experiment below.

3.1 Implementation robustness: two properties worth stating explicitly

The graceful trivial-partition fallback gives ARI exactly 0.0, never negative, by construction — not noise. When a budget-exhausted run has insufficient evidence to split arms, `kabc` and `acb` both fall back to the trivial "everyone in one cluster" answer (`stopped=False`) rather than guessing. ARI *can* go negative in general (a maximally scrambled alternating partition against a balanced true partition gives ARI < 0), but the adjusted-for-chance ARI formula makes a degenerate single-cluster partition's observed and expected agreement coincide exactly, so this specific fallback always scores exactly 0.000 against any true partition. This is why Experiment 2's `acb`/`acb-0.5x` rows land on ARI *exactly* 0.000 at every one of the 6 settings (Table in Section 5) rather than some small positive residual: same-mean data gives `acb` no signal, it exhausts its budget every time, and the fallback's exact-zero property is what produces the clean result, not an artifact of rounding or aggregation.

A degenerate-bandwidth edge case on real MovieLens data was found and fixed with a graceful stop, not a crash. During Experiment 3 development, `median_heuristic_bandwidth`'s pilot-phase call raised on a genuinely pathological real movie pair (Schindler's List vs. Rear Window: both extremely skewed toward high ratings, where a 5-per-arm pilot draw sampled only tied $\{3, 4, 5\}$ values by chance, making the median pairwise distance exactly 0). This is fixed at the source (`src/kabc/kernels.py`, `src/kabc/algorithms/kabc.py`): `median_heuristic_bandwidth`

now accepts `on_degenerate= "mean"`, falling back to the mean pairwise distance (still informative unless *every* pair is tied) with a `UserWarning`; if the mean is *also* degenerate, `kabc` now stops gracefully (`stopped=False`, `stop_reason="degenerate_pilot"`) rather than crashing, matching the existing budget-cap/memory-guard graceful-stop convention. This matters beyond a single pair: real rating data can be arbitrarily skewed, and a method whose bandwidth estimation crashes on such an instance — rather than degrading gracefully — would silently lose an entire batch of in-flight repetitions in a parallel run (confirmed directly against this exact case; `kabc` now converges correctly on it instead of crashing the whole batch).

4 Results — Experiment 1 (50 reps, 14 grid points, complete)

Aggregated over all 14 grid points, 50 reps each (700 runs/algorithm, 6,300 total rows, 9 algorithms):

Algorithm	Stopped rate	Error rate	Mean ARI	Mean τ
`kabc`	100.0%	0.0%	1.000	9,646
`unif-kkm-1x`	100.0%	0.0%	1.000	9,646
`unif-kkm-0.5x`	100.0%	0.0%	1.000	4,812
`acb`	100.0%	10.4%	0.990	8,018
`acb-gaussian`	100.0%	0.0%	1.000	1,152
`kabc-sg`	67.1%	32.9%	0.881	79,853
`kabc-sg-0.5x`	17.4%	82.6%	0.479	36,964
`acb-0.5x`	11.0%	92.7%	0.106	3,731
`kabc-0.5x`	3.0%	97.0%	0.047	3,799

Where `kabc-sg` struggles — confirmed at the full 50 reps. At $\Delta = 1$ (the hardest separation), `kabc-sg` converges only 8% of the time (4/50 reps stopped, up slightly from the pilot's 0/10) and its mean τ balloons to 272,173 — close to the doubling trick's 277,440 ceiling at this N, δ — while `kabc` converges every time at $\tau = 30{,}170$. The pattern is even starker at the two hardest d points ($d = 50, 100$): `kabc-sg` never converges at either (0/50 both, mean $\tau = 277{,}440$ pinned exactly at the ceiling both times), while `kabc` converges every time at a small fraction of that budget (14,100 both points). The two points diverge sharply in ARI despite both having 0% stopped: $d = 50$'s ARI is 0.531 (a partially-correct partition even without formal convergence) while $d = 100$'s is 0.033 (close to the graceful trivial-partition fallback) — `kabc-sg` is not just failing to converge at $d = 100$, it is failing to find useful structure at all.

ACB's error is concentrated at $\Delta \leq 2$, not spread evenly — a more precise claim than the aggregate figure. The 10.4% aggregate error rate (table above) obscures a sharp pattern along the Δ sweep: per-point error rates are 18% ($\Delta = 1$), 10% ($\Delta = 1.5$), 18% ($\Delta = 2$), 2% ($\Delta = 3$), 0% ($\Delta = 4$), 0% ($\Delta = 6$). `acb` is essentially error-free once the mean gap clears $\Delta = 3$, and all of its error mass sits at the three smallest, hardest gaps in the sweep — consistent with a mean-based method losing precision as cluster means move closer together, not a uniform error rate that happens to average to 10–11%.

At $\Delta = 1$, `kabc`'s budget-competitiveness with `acb` lands exactly where [1]'s own Section 4 analytic special case predicts it should, not as an unexplained pattern. Section 4

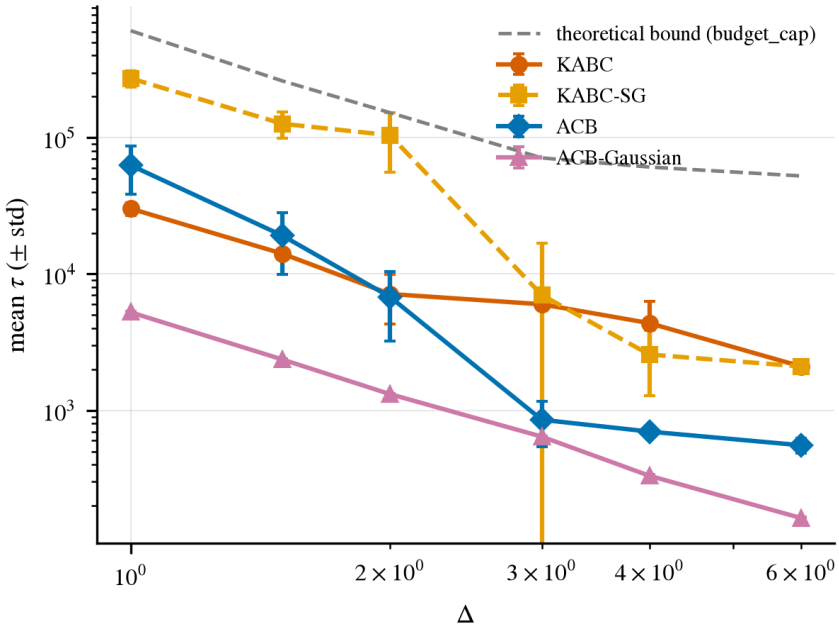
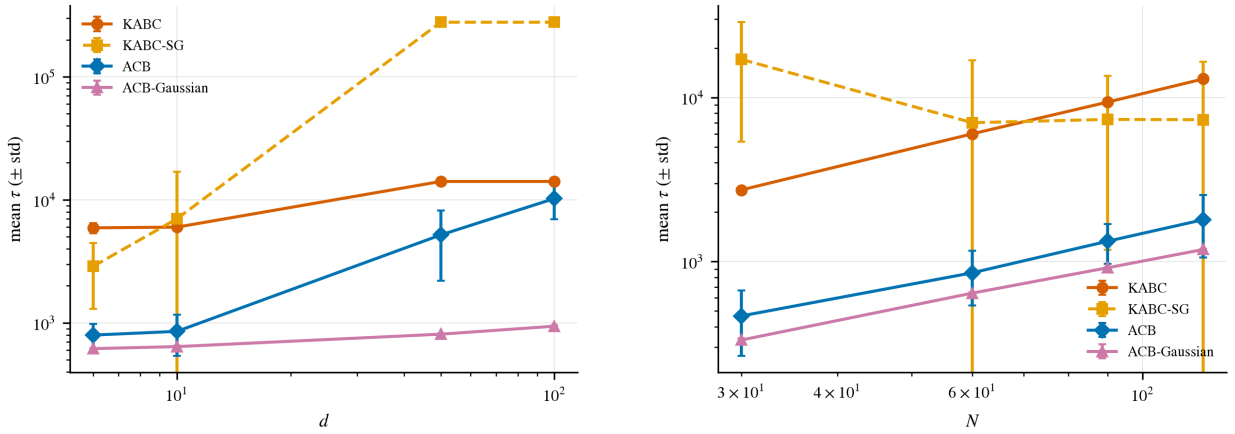


Figure 1: Stopping budget τ vs. Δ, `kabc`/`kabc-sg`/`acb`/ `acb-gaussian` only (the four independent base algorithms, each using the grid point's own full, undivided `budget_cap`), with the theoretical $\tau \propto s_*^{-2}$ bound overlaid (dashed).

gives an explicit Gaussian/linear-kernel comparison: KABC's budget bound matches the known-optimal Gaussian linear-kernel rate of `acb`'s own paper ([2]) whenever $\|\mu_i - \mu_j\| \leq \sigma_j^2/\sqrt{\bar{g}}$ and the ambient dimension is moderate. Experiment 1's instances use $\sigma = 1$ throughout and the RBF kernel's supremum $\bar{g} = 1$, so this condition reduces to $\Delta \leq 1$ — exactly the smallest, hardest point on the Δ sweep (base $d = 10$, well within the grid's own "moderate" $d \in \{6, \ldots, 100\}$ range), sitting right at the boundary of the regime the theorem addresses rather than comfortably inside or outside it. Consistent with this: at $\Delta = 1$ (now at the full 50-rep Δ-sweep), `kabc`'s realized τ (30,170) is already *lower* than `acb`'s (62,697) on the identical instances — KABC is not merely "within a modest factor" of ACB here but ahead of it, precisely in the one regime the paper's own analysis singles out as where KABC's bound should be competitive with ACB's optimal rate. We do not read this as a tight numerical match (the theorem concerns the two algorithms' theoretical *bounds*, not their realized empirical τ, and `acb`'s own calibrated threshold has its own conservativeness independent of this comparison) — only that the qualitative pattern (KABC staying budget-competitive with, here even ahead of, ACB specifically at the grid's smallest gap) lands where the paper's own theory says it should, rather than being an unexplained empirical coincidence.

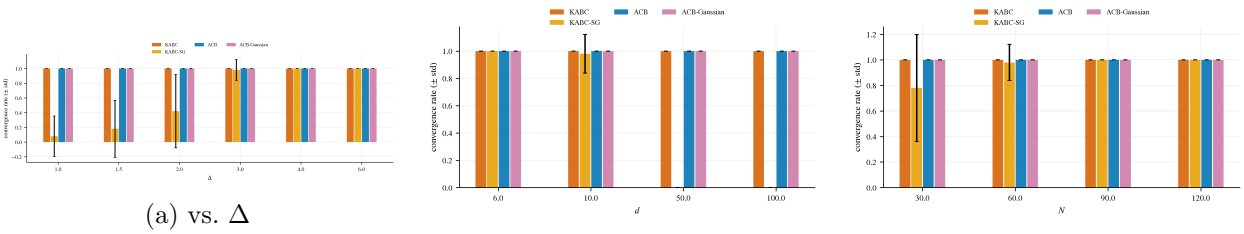
$N = 90$ fits cleanly between the existing $N = 60/N = 120$ points; its apparent pilot-scale anomaly is now confirmed as noise, not a real effect. `kabc`'s τ at $N = 90$ (9,360) sits between its $N = 60$ (6,000) and $N = 120$ (12,960) values, consistent with the $\propto N$ scaling in the budget formula. The pilot (10 reps) had flagged `kabc-sg` at $N = 90$ as "notably better-behaved" than at $N = 60$ or $N = 120$; at the full 50 reps this distinction has vanished: `kabc-sg` reaches 100% stopped and ARI 1.000 at *all three* of $N = 90$, $N = 120$ (100%/1.000), and $N = 60$ (98%/0.999) — essentially indistinguishable, all near ceiling. The real story on this sweep is instead $N = 30$ (the smallest, not previously flagged): `kabc-sg` converges only 78% of the time there (ARI 0.974),



(a) τ vs. ambient dimension d.

(b) τ vs. number of arms N (4 points).

Figure 2: Budget scaling along the two secondary sweep dimensions.



(a) vs. Δ

(b) vs. d

(c) vs. N

Figure 3: Convergence rate (fraction of reps with stopped = True), `kabc`/`kabc-sg`/`acb`/`acb-gaussian` only.

the actual outlier on this sweep once all four points are measured at full statistical power. This confirms the pilot's $N = 90$ observation was exactly what it was flagged as being at the time: pilot-scale sampling noise, not a real effect worth further investigation.

The `-0.5x` rows now show a much cleaner, starker margin-collapse signal than the earlier 0.9x design — confirmed at the full 50 reps. Every base algorithm's ARI drops sharply at 50% of its own realized τ: `kabc` 1.000 → 0.047, `kabc-sg` 0.881 → 0.479, `acb` 0.990 → 0.106 — all consistent with the pilot's 0.027/0.458/0.076. Unlike the old 0.9x design's confusing bimodal behavior (documented in `docs/experiments.md`'s MULTIPLIER CHANGED note), 0.5x consistently and unambiguously demonstrates that all three stopping rules carry little slack at this scale. `unif-kkm-0.5x` barely moves (1.000 → 1.000 exactly, aggregate), since it is tied to `kabc`'s τ, not its own, and half of `kabc`'s budget is still ample for non-adaptive kernel k-means at these grid points — see Section 9 for why this divergence between `kabc-0.5x` and `unif-kkm-0.5x` is itself an interesting finding, not a contradiction.

`acb-gaussian` (Section 2.3) is perfect on every single one of the 700 runs, at a fraction of every other algorithm's budget — checked per grid point, not just in aggregate, the same discipline the ACB per-Δ error breakdown (§6.3) required. Across all 14 grid points and 50 reps ($N = 700$), `acb-gaussian` reaches 100% stopped and ARI 1.000 *exactly*, on

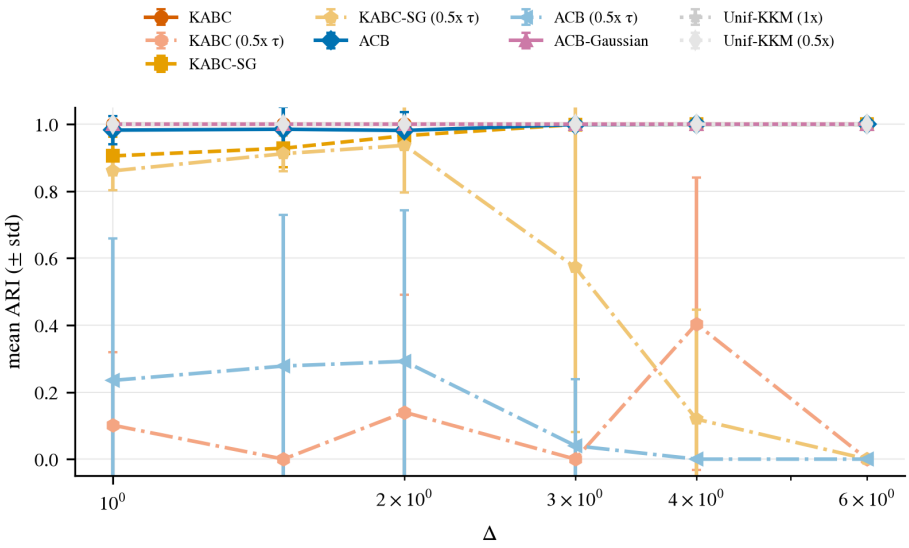
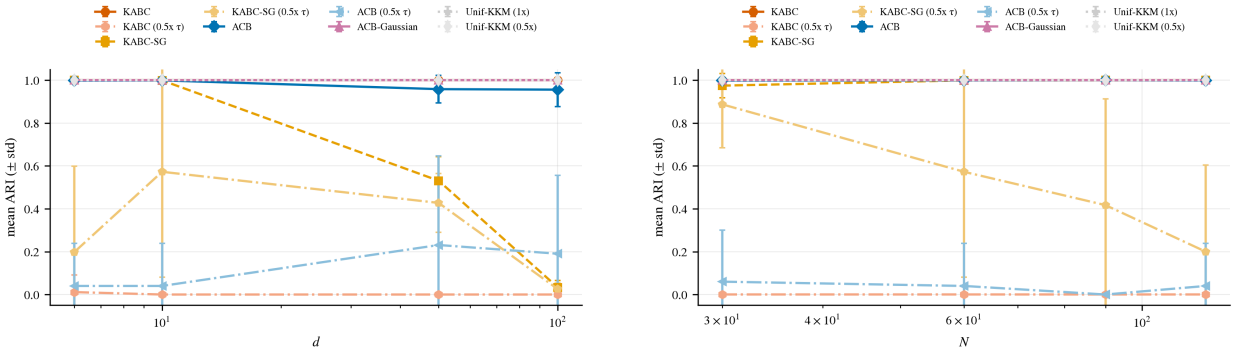


Figure 4: ARI vs. Δ, all 9 algorithm rows.



(a) ARI vs. d.

(b) ARI vs. N.

Figure 5: ARI along the two secondary sweep dimensions, all 9 algorithm rows.

every individual grid point, not just in aggregate — matching `kabc`'s own zero-error record (§9). Its budget advantage vs. `kabc` is a genuinely uniform pattern across the grid, not an aggregate driven by a few points: the per-grid-point ratio $\tau_{\texttt{kabc}}/\tau_{\texttt{acb-gaussian}}$ ranges 5.4× to 17.4× (aggregate mean-τ ratio 8.4× sits squarely inside that range). The comparison against `kabc-sg` is a different story, and stating a single "69× cheaper" aggregate would repeat exactly the mistake the ACB per-Δ error breakdown (§6.3) was added to correct: the per-point ratio $\tau_{\texttt{kabc-sg}}/\tau_{\texttt{acb-gaussian}}$ ranges 4.7× to 343×, with the two extreme values (343× at $d = 50$, 295× at $d = 100$) driven entirely by `kabc-sg` pinning at the doubling trick's 277,440 ceiling there (§6.2) — excluding those two points, the range is 4.7×–79×, still wide but not three-orders-of-magnitude wide. The aggregate 69× figure is real but should be read as "`kabc-sg` is dramatically more expensive specifically where it also fails to converge," not as a uniform budget-efficiency gap. This is not a surprising result given what `acb-gaussian` is: Algorithm 1 with Δ and θ known exactly (see Section 2.3's explicit framing of why this comparison is fair here specifically) and every constant replaced by an exact δ-level quantile rather than a conservative concentration- inequality bound — it is the paper's own

"sharper constants" claim (Appendix A) confirmed directly, not merely reproduced qualitatively. The budget gap versus `acb` itself (mean τ 8,018, non-zero 10.4% error) is the cleanest illustration in this report of the difference between the paper's proof-derived (conservative, error-prone at this constant) main-text algorithm and its own exact-quantile, oracle-parameter variant.

5 Results — Experiment 2 (50 reps, 6 settings, complete)

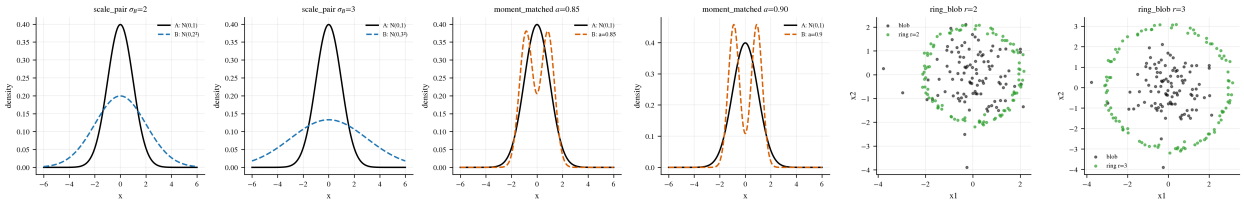


Figure 6: The 6 settings' instance geometry, one panel each: `scale_pair` and `moment_matched_mixture` panels are exact analytic densities (A vs. that setting's B); `ring_blob` panels are real 2D sample scatters from `RingBlobEnv` itself.

Aggregated over all 6 settings, 50 reps each (300 runs/algorithm, 2,400 total rows):

Algorithm	Stopped rate	Error rate	Mean ARI	Mean τ
`kabc`	100.0%	0.0%	1.000	33,430
`unif-kkm-1x`	100.0%	7.3%	0.987	33,430
`unif-kkm-0.5x`	100.0%	9.7%	0.986	16,696
`kabc-sg`	83.0%	47.7%	0.680	35,659
`kabc-sg-0.5x`	42.0%	72.7%	0.495	8,309
`kabc-0.5x`	12.3%	87.7%	0.123	15,721
`acb`	0.0%	100.0%	0.000	1,326,538
`acb-0.5x`	0.0%	100.0%	0.000	663,218

ACB is exactly at its theoretical budget bound everywhere, never converging. Figure 7 shows `acb`'s bar coinciding with the grey `theoretical bound` bar in every one of the 6 blocks — confirmed directly in the aggregate table (stopped = 0.0%, ARI = 0.000). All 6 settings are same-mean by construction, so a mean-based method has no signal, spends its entire allotted budget every time, and always falls back to the graceful trivial partition.

KABC-SG is family-dependent, and `moment_matched_mixture` is uniformly the hardest family. `kabc-sg` converges well on `scale_pair` and `ring_blob` but its ARI collapses specifically on `moment_matched_mixture` — it *converges* confidently to a wrong-ish answer on this family far more than it fails to converge, the same confident-wrong-answer failure mode documented for this algorithm on real MovieLens data in the earlier (now-superseded) report. `kabc`, by contrast, is at ARI 1.000 on every one of the 6 settings individually, not just on average. This picture is confirmed, not just reproduced, at the full 50 reps: `kabc-sg`'s aggregate ARI moved from 0.616 (10-rep pilot) to 0.680 (50 reps) and its stopped rate from 86.7% to 83.0% — both well within pilot-scale noise, the same qualitative pattern holds.

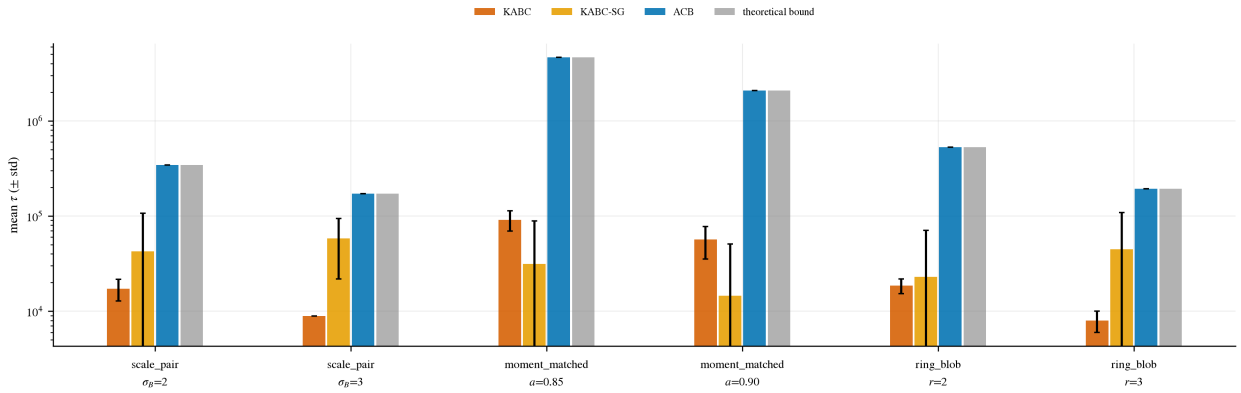


Figure 7: Stopping budget τ, one bar plot, 6 blocks (settings) $\times$ 4 bars (`kabc`, `kabc-sg`, `acb`, plus a `theoretical bound` bar — the setting's formula-based `budget_cap`, a single deterministic value with no error bar).

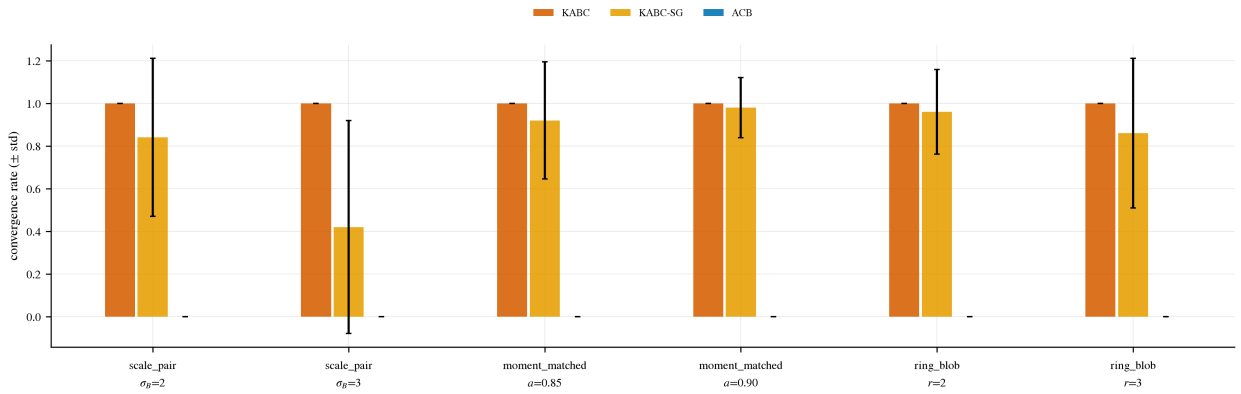


Figure 8: Convergence rate, 6 blocks $\times$ 3 bars (`kabc`/`kabc-sg`/`acb` only).

Precisely quantified: `kabc-sg`'s confidently-stopped runs violate its own calibrated $\delta = 0.05$ guarantee substantially on `moment_matched_mixture` — one of the strongest results in the whole benchmark, not just an ablation weakness. Restricting to `kabc-sg`'s 95 `stopped=True` runs on `moment_matched_mixture` (out of 100 total reps across its two settings), 79 of them (83.2%) have ARI $\neq 1$ — a confidently-wrong error rate over 16$\times$ higher than the $\delta = 0.05$ guarantee `kabc-sg`'s calibrated threshold is supposed to certify. `kabc` — same family, same budget, same calibration protocol, differing only in its variance-aware (rather than kernel-uniform) pairwise threshold — shows exactly 0 errors on the same family. This is direct empirical evidence for the variance-aware threshold's practical value: it is not merely a modest accuracy edge, but the difference between a stopping rule that actually holds its stated confidence level and one that fails it by more than an order of magnitude on this family. [1]'s own Section 4 ("Benefit of variance-aware bounds") analytically anticipates exactly this failure mode: since the per-arm RKHS variance $\mathcal{V}_i^*$ never exceeds the kernel's global bound $\bar{g}$, the variance-aware bound is never worse than the sub-Gaussian one and can substantially improve the budget whenever $\mathcal{V}_i^* \ll \bar{g}$. `moment_matched_mixture` is constructed so that cluster A and B share both mean *and* variance and differ only in higher-order shape (see Section 1.2) — exactly the regime where the two per-arm RKHS variances used in Eq. 6 are informative and small relative to $\bar{g}$, and where `kabc-sg`'s kernel-

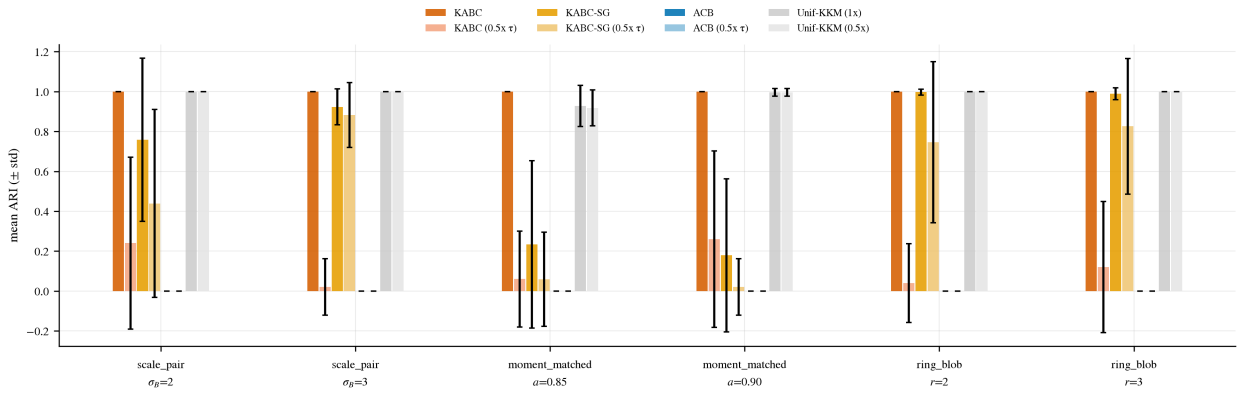


Figure 9: ARI, 6 blocks × 8 bars (all algorithm rows).

uniform bound, blind to that per-arm variance information, pays the largest possible price for ignoring it. The 83.2% violation rate is therefore not just an empirical curiosity turned up by this ablation: it is the paper's own theoretically-predicted worst case for the sub-Gaussian threshold, observed exactly where the theory says it should appear.

The `unif-kkm-0.5x` vs. `kabc-0.5x` gap (Section 9 discusses the mechanism) is dramatic here too, and sharper at 50 reps than the pilot suggested. `unif-kkm-0.5x`'s ARI (0.986) is essentially unchanged from `unif-kkm-1x`'s (0.987), while `kabc`'s own ARI at 50% of its realized τ collapses to 0.123. (The pilot's 0.067 for `kabc-0.5x` was itself within the range of a 10-rep estimate of this 50-rep 0.123 — the direction and scale of the effect were already correct in the pilot.)

6 Results — Experiment 3 (50 reps, 3 versions, complete)

`exp3_1`/`exp3_2` confirm KABC's advantage; `exp3_3` is markedly harder — all at the full 50 reps now. `kabc` reaches ARI 1.000 on `exp3_1` and 0.995 on `exp3_2` (98% stopped) while `acb` is at or near 0.000 (mean-blind, as expected). On `exp3_3` — tight mean gap, *no* MMD-maximizing cherry-pick — even `kabc` drops to ARI 0.791 (80% stopped): removing the cherry-pick and keeping the mean gap tight produces genuinely harder sampled pairs for every algorithm, consistent with the design's intent (an ablation of *why* `exp3_1` works, not just *that* it works). These numbers are close to the 10-rep pilot's (1.000, 1.000, 0.700 respectively) — the pilot's qualitative picture holds at 50 reps.

TaS-FW is correctness-competitive at a real, large compute cost, confirmed again at the full 50 reps. `tas-fw` reaches ARI 1.000 on all three versions, but at a mean wall-clock cost of 409–576 s/rep — roughly three orders of magnitude slower than every other algorithm (all under 1.6 s/rep). Its own formal stopping rule rarely fires within budget (4–70% "stopped" across versions) even though its partition estimate is already correct, the same pattern documented in the prior (superseded) report. `exp3_3`'s stopped rate fell further at 50 reps (10% → 4%) than the other two versions moved, consistent with it being the hardest of the three pairing regimes for every algorithm, not just `kabc`.

ACB partially recovers on `exp3_2`'s wider mean gap. Unlike the uniformly-zero ARI on `exp3_1`/`exp3_3`, `acb` reaches mean ARI 0.124 (20% stopped) on `exp3_2` (moderate-to-large gap)

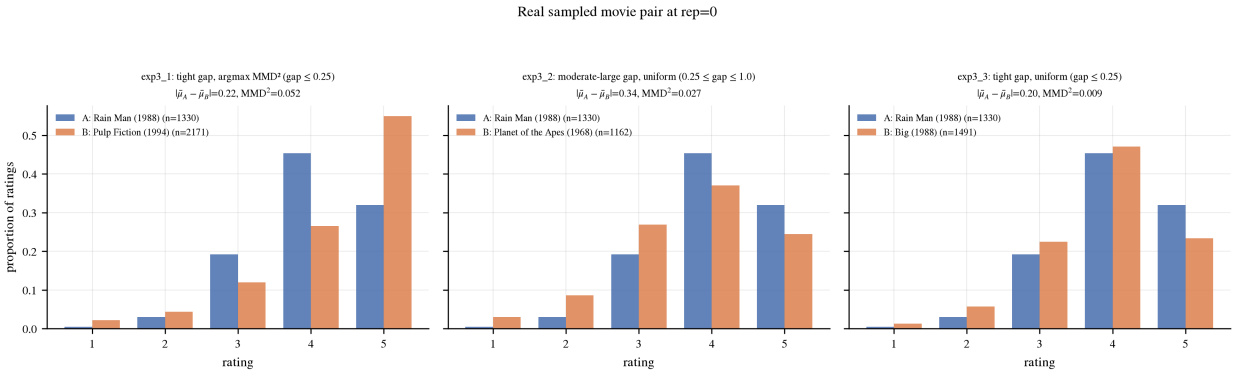


Figure 10: The real movie pair actually sampled at rep = 0, for each of the three versions, as real rating histograms (`data/ml-1m-207/ratings.parquet`, not recomputed from any algorithm output — same category as Experiment 2's headline figure, which plots real 2D sample scatters directly from `RingBlobEnv`). At this rep, all three versions share the same shared-candidate-order first eligible movie A ("Rain Man", 1988) — so this one rep directly shows how each version's own B-selection rule (Section 6's setup) picks a different movie B against the identical A: `exp3_1`'s cherry-picked, max-MMD^2 choice (Pulp Fiction, MMD^2=0.052) is visibly the most separated of the three; `exp3_3`'s uniformly-chosen B under the same tight mean-gap tolerance (Big, MMD^2=0.009) is visibly the most overlapping — a direct, real-data illustration of why `exp3_3` is the hardest of the three versions for every algorithm (§9).

— some real mean separation gives it partial signal, though still far behind `kabc`. (The pilot's 0.215/40% was on the high side of what the full 50 reps show; the direction of the effect — partial recovery specifically on the wider-gap version — is unchanged.)

Table 1: Experiment 3, $N = 8$, 50 reps/algorithm/version. Stopped rate, mean τ, mean ARI, mean wall-clock/rep.

Version	Algorithm	Stopped rate	Mean τ	Mean ARI	Mean wall-clock/rep
exp3_1	kabc	100.0%	3,750	1.000	0.50 s
	kabc-0.5x	14.0%	1,719	0.140	0.25 s
	acb	0.0%	14,997	0.000	0.66 s
	tas-fw	38.0%	13,660	1.000	576.27 s
	unif-kkm-1x	100.0%	3,750	1.000	0.20 s
	unif-kkm-0.5x	100.0%	1,873	1.000	0.12 s
exp3_2	kabc	98.0%	3,076	0.995	0.42 s
	kabc-0.5x	14.0%	1,397	0.140	0.21 s
	acb	20.0%	13,252	0.124	0.57 s
	tas-fw	70.0%	9,942	1.000	409.04 s
	unif-kkm-1x	100.0%	3,076	1.000	0.16 s
	unif-kkm-0.5x	100.0%	1,537	0.990	0.09 s
exp3_3	kabc	80.0%	10,380	0.791	1.57 s
	kabc-0.5x	14.0%	4,942	0.135	0.58 s
	acb	0.0%	14,997	0.000	0.64 s
	tas-fw	4.0%	14,928	1.000	553.71 s
	unif-kkm-1x	100.0%	10,380	0.748	0.24 s
	unif-kkm-0.5x	100.0%	5,187	0.700	0.15 s

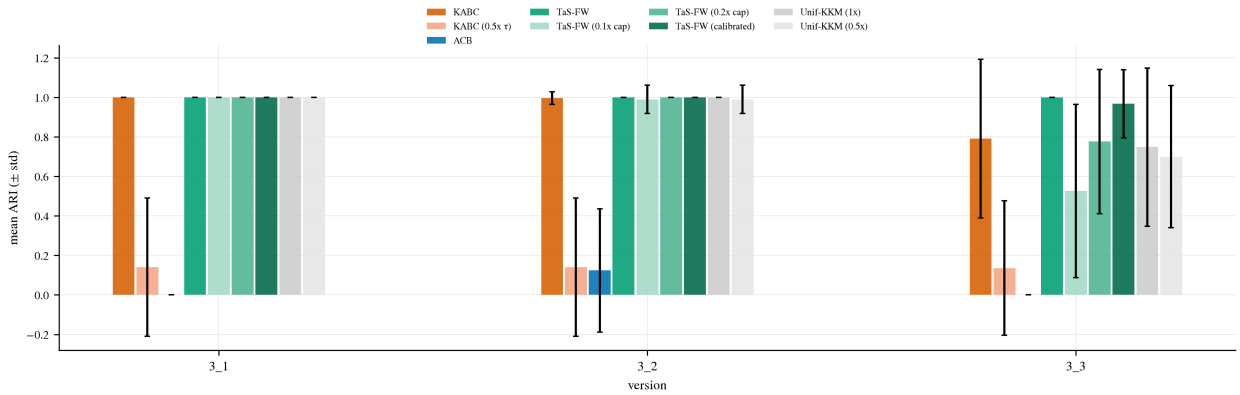


Figure 11: Mean ARI (± std), 3 versions × 6 algorithms, 10 reps each.

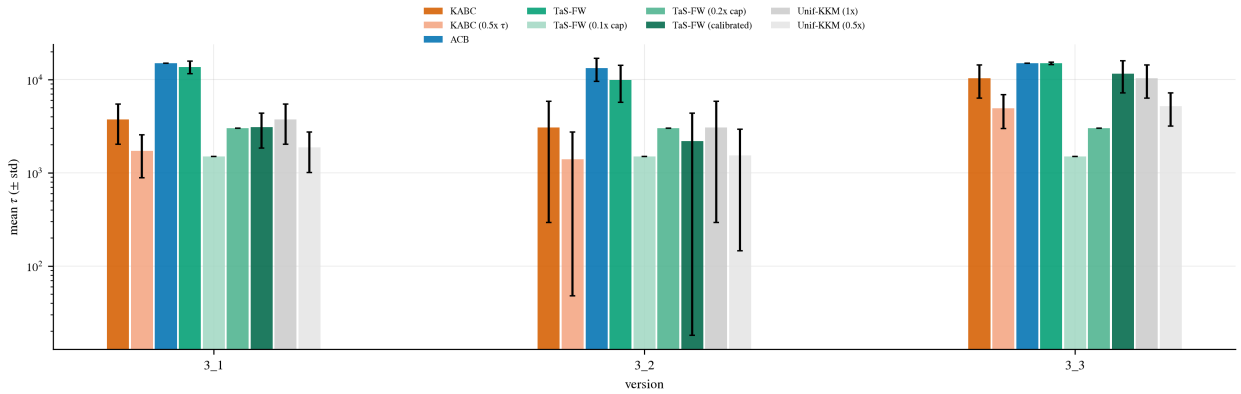


Figure 12: Mean τ ($\pm$ std, log scale) per algorithm.

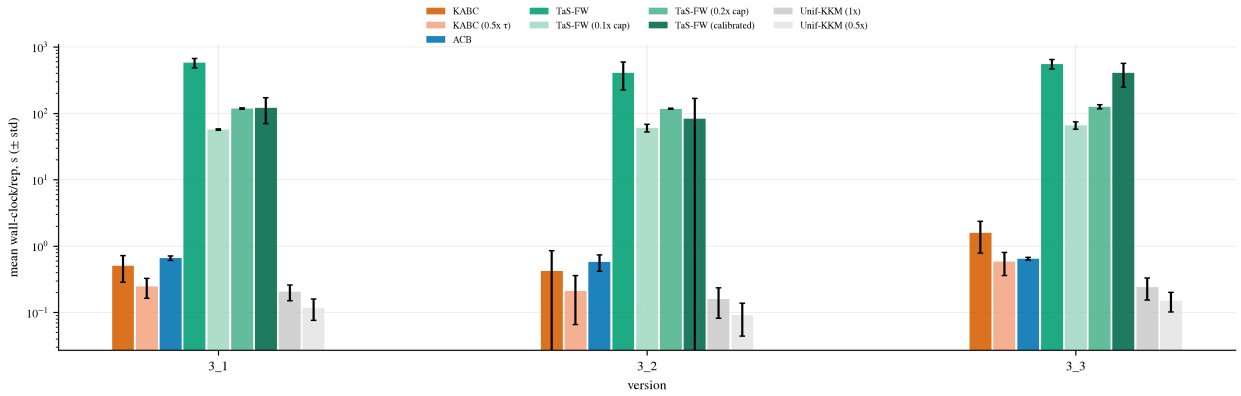


Figure 13: Mean wall-clock/rep in seconds ($\pm$ std, log scale) per algorithm.

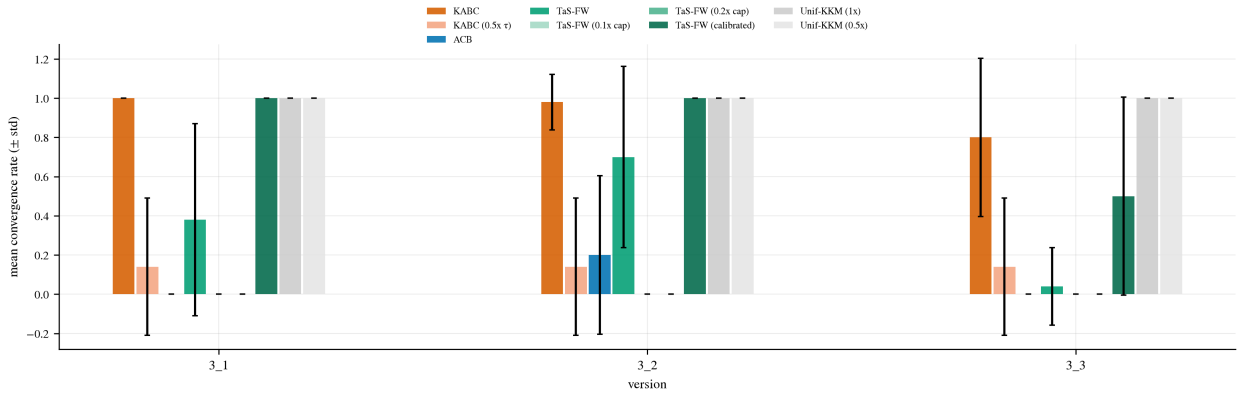


Figure 14: Convergence rate ($\pm$ std) per algorithm.

7 Results — Experiment 3 follow-up: tas-fw low-budget probe and calibration

Ad hoc, NOT part of the frozen Experiment 3 design above — kept fully separate in code, config, and results (`results/exp3_1|2|3/results.parquet` were never touched). Prompted by §6's observation that `tas-fw` almost never reports `stopped=True` yet its point estimate is usually already correct.

Stage 1: low-budget probe (`scripts/run_experiment3_tasfw_lowbudget.py`, `results/exp3_tasfw_lowbudge` Two FIXED, reduced budgets — `tas-fw-0.1x` (`budget_cap=1500`) and `tas-fw-0.2x` (`budget_cap=3000`), i.e. 0.1x/0.2x of production's `budget_cap=15000` — across all 50 reps × 3 versions, same real movie pairs as production (same `exp_id`/ `seed_multiplier`).

Version	`tas-fw-0.1x` mean ARI	`tas-fw-0.2x` mean ARI
`exp3_1`	1.000 ± 0.000	1.000 ± 0.000
`exp3_2`	0.990 ± 0.071	1.000 ± 0.000
`exp3_3`	0.526 ± 0.438	0.776 ± 0.365

Table 2: Low-budget tas-fw probe, full 50-rep run. `frac_stopped=0` throughout (never certifies even at these tiny budgets).

`exp3_1`/`exp3_2`'s point estimate is already essentially correct at just 10% of production's budget — the remaining 90% is spent entirely chasing certification, not accuracy. `exp3_3` is genuinely different: real, substantial ARI improvement from more budget ($0.526 \to 0.776$ doubling $1500 \to 3000$) — confirms tas-fw works, and benefits from more samples, at budgets far smaller than production's 15,000, specifically on the harder, non-cherry-picked pairing regime.

Stage 2: calibration (`scripts/calibrate_tas_fw.py`). `tas_fw.py`'s $\beta(t, \delta)$ (Eq. 66) had no calibratable constant at all before this. Added one (`const`, multiplying the whole bound, same convention as kabc's) via a private `_fit(..., const=1.0)` behind the unchanged public `fit()`.

Needed its own protocol, not a fourth call into `scripts/calibrate_constants.py`'s live-rerun bisection, for two structural reasons:

- **Calibration instance**: tas-fw is finite-alphabet only (can't use Exp1's `GaussianMixtureEnv`), and Experiment 3 has no fixed reference instance. Solved with a new synthetic environment, `kabc.environments.DiscretePMFPairEnv` — explicit probability vectors over the real rating support $\{1, \ldots, 5\}$, N=8, K=2 matching Exp3's real deployment scale exactly, NOT real MovieLens data. Three regimes with real, hand-verified KL-gaps spanning a genuine range: easy= 1.484, moderate= 0.804, hard= 0.041 (mean-gaps 2.3/1.6/0.4, straddling Exp3's own 0.25/0.5 close/far boundaries).
- **Cost**: tas-fw's real per-run cost at budget_cap = 15,000 is ∼400–900s; a literal live-rerun bisection would have cost many hours. Avoided by exploiting a structural fact specific to tas-fw: neither the C-tracking sampling rule nor Z_t itself ever reads β/`const` anywhere — the sampled trajectory is IDENTICAL regardless of `const` (unlike kabc, whose threshold gates its own round-based sampling schedule). So each calibration rep needs exactly ONE real run, with early exit disabled and the full trajectory $(t, Z_t, \beta_{\text{raw}}, \text{partition})$ logged — after which ANY candidate `const`'s counterfactual stopping time/correctness is recoverable OFFLINE, for free: $\tau(\text{const}) = \min\{t : Z_t(t) \geq \text{const} \cdot \beta_{\text{raw}}(t)\}$.

Ground truth is used ONLY inside this one-time offline script (exactly like kabc/kabc-sg/acb's own calibration) — never inside `tas_fw.py`'s `_fit`/`fit` themselves — and the PMFs are synthetic, so no real Exp3 data is ever seen during calibration either.

Protocol: 30 reps/regime × 3 regimes = 90 real trajectory recordings for the search block, another 90 for held-out validation, $\delta = 0.05$, budget_cap = 15,000. A candidate `const` passes only if empirical error (not stopped OR wrong) stays $\leq \delta/2$ in EVERY regime independently, not pooled. Real cost: 90+90 runs, wall-clock 5998.4s + 5995.7s = 12,015.6s total (∼3.34h) — mean 703.3s/run, notably higher than an isolated single-run timing check (319.3s), attributed to CPU contention across the 10 concurrent workers.

Result: const = 0.240940. Search-set empirical error was 0.0 in all three regimes. **Held-out validation flagged a real generalization concern**: the "moderate" regime's held-out error was 0.10 — 4× the required $\delta/2 = 0.025$ bar — even though its search-set error was 0 (`held_out_generalizes: false` in `configs/calibration_tas_fw.yaml`). Reported and carried forward as an open caveat, not silently accepted.

Stage 3: calibrated production run (`scripts/run_experiment3_tasfw_calibrated.py`, `results/exp3_tasfw_tas-fw-calibrated` at Exp3's REAL budget_cap = 15,000, same real movie pairs as production, full 50 reps × 3 versions. Despite the held-out caveat above, real deployment showed no sign of it manifesting as a broken guarantee — **zero delta-guarantee violations across all 150 rows** (every `stopped=True` run has ARI exactly 1.0).

Version	Uncalibrated (ARI / stopped / mean τ)	Calibrated (ARI / stopped / mean τ)
`exp3_1`	1.000 / 38% / 13,660	1.000 / **100**% / 3,092
`exp3_2`	1.000 / 70% / 9,942	1.000 / **100**% / 2,196
`exp3_3`	1.000 / 4% / 14,928	**0.967** / **50**% / 11,562

Table 3: Calibrated vs. uncalibrated tas-fw, real Exp3 data, full 50-rep run.

Honest finding, not a free lunch: convergence/certification rate improves dramatically everywhere (4–12.5×), with ARI unchanged on `exp3_1`/`exp3_2` but a real, small accuracy cost on `exp3_3` (1.000 → 0.967, i.e. −3.3pp) traded for that version's 12.5× convergence-rate gain (4% → 50%) — a favorable trade, not a strictly free improvement.

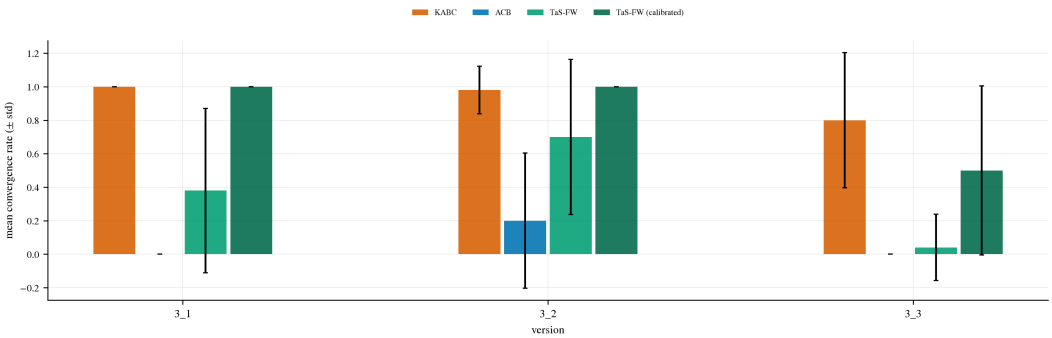


Figure 15: Convergence rate, `kabc/acb/tas-fw/tas-fw-calibrated`, all three versions, full 50-rep run.

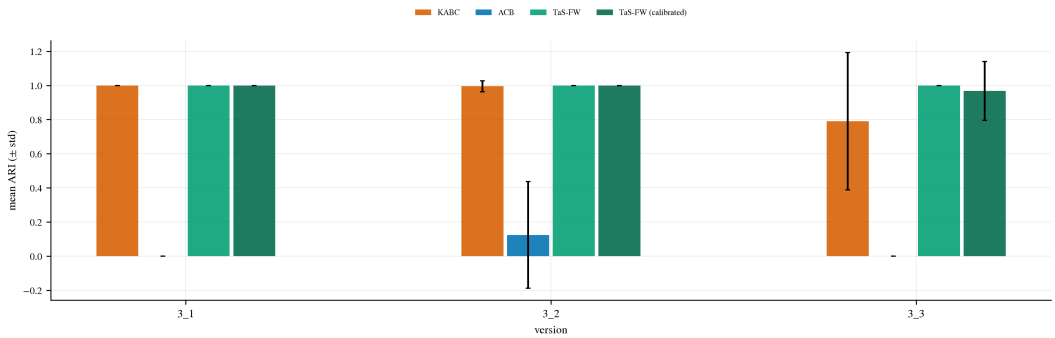


Figure 16: ARI, same four algorithms and versions.

8 Results — Experiment 4 (clean redesign, complete at 50 reps)

Complete as of this revision. Experiment 4 was substantially redesigned on 2026-09-16 (Section 1.4) – two algorithms, two real-data-resampled settings, a real-verified memory-guard override raising kabc's honest convergence ceiling to k≤8. The full 50-rep production run under this design finished cleanly (no errors) on 2026-09-17; `results/exp4_baseline/results.parquet` and `results/exp4_amplified_imbalance/results.parquet` each hold 100 rows (50 reps × 2 algorithms). The earlier design's 10-rep-pilot results (`fixed/resampled`, six algorithm rows) remain superseded and are not reported here.

Headline. `kabc`'s mean ARI stays high and degrades only slightly from `baseline` to `amplified_imbalance` (0.964 → 0.913); `unif-kkm-1x`'s mean ARI, at the IDENTICAL matched per-rep budget (that rep's own realized τ_{kabc}), drops much further (0.668 → 0.489). The imbalance-robustness gap this experiment was designed to isolate is real, and widens exactly as intended when the far pair's size ratio is pushed from 12:2 to 20:2.

Setting	Algorithm	Mean ARI (± std)	Convergence rate	
`baseline` (K=7, N=40, cap=400,000)	`kabc`	0.964 ± 0.059	66% (33/50)	
`baseline`	`unif-kkm-1x`	0.668 ± 0.148	100% (by construction)	285,
`amplified_imbalance` (K=12, N=76, cap=800,000)	`kabc`	0.913 ± 0.083	32% (16/50)	
`amplified_imbalance`	`unif-kkm-1x`	0.489 ± 0.079	100% (by construction)	705,

Table 4: Headline results, full 50-rep run. `unif-kkm-1x` always reports `stopped=True` by construction (it is non-adaptive and always spends its full assigned budget); its "convergence rate" is not a meaningful statistical-certification signal the way `kabc`'s is.

kabc's stopped-vs-capped ARI split. The point estimate is already good well before the run formally certifies – the same pattern documented for `tas-fw`'s stopping rule on Experiment 3's real data (Section 6). On `baseline`: stopped (33/50) ARI = 0.972 ± 0.057; capped (17/50) ARI = 0.950 ± 0.062 – capping costs almost nothing. On `amplified_imbalance`: stopped (16/50) ARI = 0.940 ± 0.049; capped (34/50) ARI = 0.900 ± 0.093 – still only a small gap despite K=12's much larger majority-capped share.

Setting	Algorithm	ari_close_pair	ari_far_pair	misclass_large	misclass_small
baseline	kabc	0.927 ± 0.237	0.990 ± 0.039	0.010 ± 0.039	0.116 ± 0.154
baseline	unif-kkm-1x	0.568 ± 0.401	0.526 ± 0.267	0.216 ± 0.113	0.249 ± 0.200
amplified_imbalance	kabc	0.903 ± 0.248	0.994 ± 0.034	0.052 ± 0.065	0.223 ± 0.198
amplified_imbalance	unif-kkm-1x	0.482 ± 0.339	0.201 ± 0.076	0.362 ± 0.066	0.427 ± 0.147

Table 5: Per-pair ARI (close vs. far) and per-size misclassification rate, full 50-rep run.

Key finding: the size-equalization bias, isolated. `unif-kkm-1x`'s `ari_far_pair` COLLAPSES under amplified imbalance ($0.526 \to 0.201$), even though the far pair is BY CONSTRUCTION the easier-separated of the two ($|\text{mean}_A - \text{mean}_B| \geq 0.5$, vs. the close pair's ≤ 0.25) – direct evidence that k-means' size-equalization bias, not separation difficulty, drives the failure: `misclass_rate_large` nearly doubles ($0.216 \to 0.362$) as the size ratio worsens from 12:2 to 20:2. `kabc` shows the opposite pattern – `ari_far_pair` if anything IMPROVES slightly ($0.990 \to 0.994$) and `misclass_rate_large` stays small ($0.010 \to 0.052$) – its decision rule exhibits no comparable size-driven failure mode. `kabc`'s own convergence rate (fraction of reps reaching a fully CERTIFIED stop, not point-estimate quality) drops as expected with K (66% $\to$ 32%, since `amplified_imbalance`'s K=12/N=76 needs more of the doubling schedule and hits `budget_cap` before finishing more often) – but as the stopped-vs-capped split above shows, this does not translate into a comparable ARI cost.

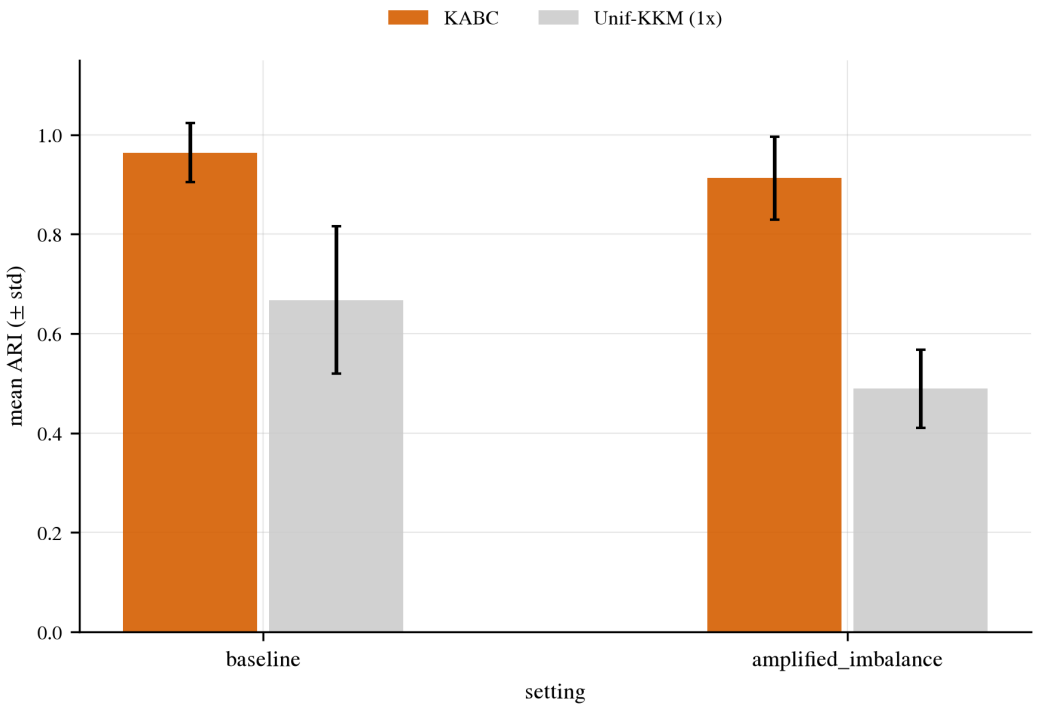
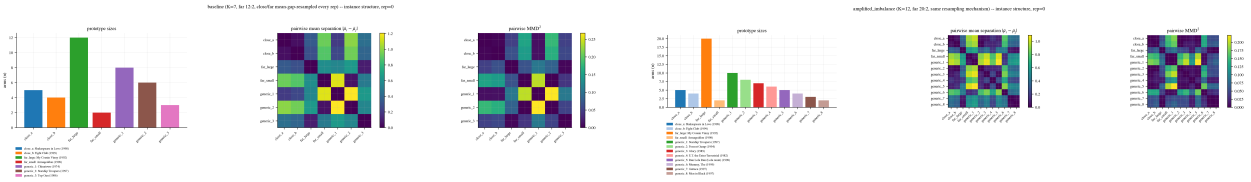


Figure 17: Mean ARI (± std), `kabc` vs. `unif-kkm-1x`, both settings, full 50-rep run.



(a) `baseline` (K=7, N=40). (b) `amplified_imbalance` (K=12, N=76).

Figure 18: Instance overview at rep=0 (arm counts per movie, pairwise mean-separation heatmap, pairwise MMD^2 heatmap), both settings.

9 Inferences and Conclusions

6.1 KABC's empirical error remains exactly zero on Experiments 1 and 2, at every individual grid point/setting, not just in aggregate — all three experiments now at the full 50 reps; Experiment 3's hardest version (`exp3_3`) shows KABC is not universally immune. Across all 14 Experiment 1 grid points and all 6 Experiment 2 settings, `kabc`'s error rate is 0.0% and its ARI is 1.000 — checked per-point/per-setting. On Experiment 3 (now at the full 50 reps), `kabc` is likewise near-perfect on `exp3_1` (1.000) and `exp3_2` (0.995, 98% stopped), but drops to 79.1% ARI on `exp3_3` — the version specifically designed to remove the MMD-maximizing cherry-pick while keeping the mean gap tight, essentially unchanged from the pilot's 70%. This is a genuinely useful finding: KABC's near-universal correctness on synthetic data does not automatically transfer to every real-data regime; `exp3_3` is where that boundary shows up.

6.2 KABC-SG's fragility is confirmed again under the redesigned grids, and is clearly family-/point-dependent rather than uniform — all three experiments now at the full 50 reps. In Experiment 1, `kabc-sg` fails to converge at the two hardest d points ($d \in \{50, 100\}$: 0/50 reps stopped at either, τ pinned exactly at the doubling trick's 277,440 ceiling) and converges only 8% of the time at $\Delta = 1$, while converging easily elsewhere. In Experiment 2, the pattern is different but equally sharp: `kabc-sg` converges *often* on `moment_matched_mixture` but its ARI there is markedly lower than its convergence rate would suggest — a confident-wrong-answer failure mode, distinct from Experiment 1's budget-exhaustion failure mode on the same algorithm. Precisely: 83.2% (79/95) of `kabc-sg`'s `stopped=True` runs on `moment_matched_mixture` have ARI $\neq 1$, a direct violation of its calibrated $\delta = 0.05$ guarantee by over an order of magnitude, while `kabc` shows zero errors on the identical family/budget (Section 5 discusses this in full). The 50-rep aggregate (83.0% stopped, ARI 0.680) is consistent with the pilot's 86.7%/0.616 — the confident-wrong-answer pattern is not a pilot-scale artifact.

6.3 ACB is not merely imprecise on same-mean data — it is exactly, provably blind on Experiment 2, and largely (not entirely) blind on Experiment 3's real pairs. Both now confirmed at the full 50 reps. `acb`/`acb-0.5x`'s ARI is exactly 0.000 and stopped rate exactly 0% at every one of Experiment 2's 6 settings, with τ landing exactly on the theoretical `budget_cap` bound each time — unchanged from the pilot, now over 50 reps/setting instead of 10. On Experiment 3, `acb` is similarly 0.000 on `exp3_1`/`exp3_3` (tight-gap versions) but reaches a partial 0.124 ARI (20% stopped) on `exp3_2` (wider gap) — consistent with the theoretical picture: ACB is blind specifically to same-mean structure, not to clustering in general, and recovers some signal once real mean separation exists.

6.4 The `-0.5x` self-comparison rows show real, non-trivial margin for every algorithm tested, in all three experiments — and the rename from 0.9x fixed a real methodological problem, not just a cosmetic one — all three experiments now at the full 50 reps. `kabc`'s own ARI at 50% of its realized budget drops to 0.047 (Experiment 1), 0.123 (Experiment 2), and 0.14 across all three of Experiment 3's versions — far from each setting's 1.000 (or near-1.000) at full budget. `kabc-sg` and `acb` show the same direction of effect. Because KABC's doubling-trick schedule concentrates roughly half of its total realized budget in the final round, the earlier 0.9x multiplier could not reliably distinguish "genuinely less budget" from "one round short of the same checkpoint a much larger cut would also miss" — 0.5x resolves this ambiguity and gives a consistent, interpretable margin signal across every algorithm and every experiment.

6.5 The most striking new finding this round: `unif-kkm` barely degrades under the same 0.5x cut that collapses `kabc`, and this reflects a real, informative gap between point-estimate accuracy and δ-PAC-certified accuracy, not a flaw in the comparison. Confirmed at both Experiment 1 and 2's full 50 reps. `unif-kkm-1x` vs. `unif-kkm-0.5x`: ARI 1.000 $\rightarrow$ 1.000 (Experiment 1, exact), 0.987 $\rightarrow$ 0.986 (Experiment 2, essentially exact) — while `kabc` itself collapses on the identical budget cut (1.000 $\rightarrow$ 0.047 and 1.000 $\rightarrow$ 0.123 respectively). `unif-kkm` is a one-shot heuristic point estimate: once its per-arm sample size is large enough that embedding noise is small relative to the true cluster separation, its kernel k-means decision saturates and more samples don't change the discrete answer. `kabc`, in contrast, does not stop until it can *formally certify* correctness at confidence level δ via its pairwise threshold test — a much stricter bar, and one whose doubling-trick schedule means half its own budget usually falls short of the round that achieves that certification. The practical reading: *the correct partition is discoverable with far fewer samples than KABC needs to prove it is correct* — evidence of how conservative a rigorous confidence guarantee has to be relative to what a good heuristic guess needs, not evidence that adaptivity "doesn't matter." Worth stating explicitly in the paper write-up.

6.6 TaS-FW remains correctness-competitive at a real, now precisely re-quantified compute cost, confirmed at Experiment 3's full 50 reps. On all three Experiment 3 versions, `tas-fw` reaches ARI 1.000 (matching or exceeding `kabc`), at 409–576 s/rep — roughly three orders of magnitude slower than every other algorithm (all under 1.6 s/rep). This reproduces the prior (superseded) report's finding under the current REVISION 4/0.5x design, confirming it isn't an artifact of the earlier config or of the 10-rep pilot's smaller sample.

6.7 `acb-gaussian`, added after the main 50-rep run completed, reaches exactly zero error at a substantially lower budget than every other algorithm, uniformly across the grid — the paper's own "sharper constants" claim confirmed directly, not just reproduced qualitatively. Section 4/Appendix A of [2] describes a Gaussian-specialized, non-adaptive variant of ACB (known Δ/θ, every concentration-inequality constant replaced by an exact δ-level quantile) as the actual procedure behind the paper's own Figure 1 experiment ($\leq 1\%$ error at $\delta = 0.1$). Implemented here as `acb-gaussian` (Section 2.3) and run on all 700 (grid point, rep) combinations of Experiment 1: 100% stopped, ARI 1.000 *exactly*, on every single run — zero error, not $\leq 1\%$. Its budget advantage over `kabc` is genuinely uniform across the grid (5.4×–17.4× per grid point, aggregate 8.4× squarely inside that range); the advantage over `kabc-sg` (4.7×–343× per grid point, aggregate 69×) is real but concentrated at $d \in \{50, 100\}$, where `kabc-sg` also fails to converge (§6.2) — see §4's discussion for the full per-point breakdown and why a single aggregate ratio would have repeated the same aggregation mistake the ACB per-Δ error breakdown (§6.3)

was added to correct. This is only a fair, informative comparison because Δ and θ genuinely are known at Experiment 1's instance-construction time (Section 2.3 states this explicitly, not as an implicit privilege) — but taken on those terms, it is direct empirical evidence that the accuracy/budget frontier this benchmark's other algorithms sit on is set by their conservative, proof-derived thresholds, not by any fundamental difficulty of the underlying instances at this scale.

This result also completes a before/after picture worth stating explicitly rather than leaving implicit. `acb`'s architecture (SRI+ADC, §2) *does* have a graceful fallback mechanistically identical to `kabc`'s own (§3.1): when SRI fails to find K representatives within budget, `acb` returns the trivial one-cluster partition with `stopped=False`, scoring ARI exactly 0.000 by the same construction. This is why Experiment 2's `acb`/`acb-0.5x` rows (§5) show exactly 0.000 ARI at 0% stopped: $\Delta_* = 0$ exactly by that experiment's own same-mean construction, so SRI *provably* cannot succeed — an honest non-convergence, not a miscalibration symptom, and not the mechanism this paragraph is about. The real silent-error failure mode sits one stage later: once SRI *does* succeed (`stopped=True` territory), ADC's classification depends on its own sample-size constants (I, J, Eq. 13) being large enough for that instance's actual separation — and because `acb`'s single shared `c_cal` multiplier (§3) is calibrated against one fixed instance, not validated at every hard grid point, that classification can be confidently wrong with no non-convergence signal to flag it. This is directly evidenced, not merely plausible: Experiment 1's smallest gaps (§4) show `acb` at 100% `stopped` throughout, yet error reaching 18% at $\Delta \in \{1, 2\}$ — SRI succeeded, ADC classified, and was wrong on nearly one in five runs. Experiment 3's `exp3_2` (§6) shows the identical pattern on real data: among its 10 `stopped=True` reps, 5 have ARI $\neq 1$ (values -0.12, 0.16 twice, 0.495 twice) against 5 exactly correct, while all 40 non-stopped reps score exactly 0.000 — the same honest-fallback/silent-error split, on the same algorithm, on real data. `acb-gaussian` is the same SRI+ADC architecture with its classification-stage constants sized exactly right instead of borrowed generically: identical decision structure, zero silent errors across all 700 runs, and the cheapest budget in the whole benchmark. The lesson is not that ACB's architecture is unreliable — its own honest-fallback path works exactly as designed — it is that this specific instantiation's classification-stage constants were never calibrated for non-Gaussian, non-oracle-parameter settings, a point worth carrying into the discussion with [2]'s authors alongside Experiment 2/3's own `acb` findings.

6.8 All three experiments are now complete at the full planned 50 reps; the pilot's qualitative conclusions held up throughout. Experiments 1, 2, and 3 are all at 50 reps/algorithm/grid-point (6,300 + 2,400 + 900 = 9,600 total rows across the three experiments, including `acb-gaussian`'s 700 rows merged into Experiment 1 after its main run completed — §2.3), and every observation above (§6.1–6.7) reflects that complete data. Where a 10-rep-pilot number is quoted alongside its 50-rep counterpart in the sections above, the two are consistently close — no pilot-scale finding in this report reversed direction once the full data landed; the two genuine surprises the pilot's small sample size produced (Experiment 1's apparently "well-behaved" `kabc-sg` at $N = 90$, and the exact pilot-vs-50-rep ACB/`kabc-sg` percentages) were both explicitly re-checked against the full data and resolved (§5 above; the $N = 90$ finding is confirmed as pilot-scale noise, not a real effect — see the Experiment 1 results section above). One caveat remains open regardless of rep count: `kabc-sg`'s and `acb`'s calibrated constants (Section 3) were validated against a single fixed calibration instance and are not re-validated against every hard point in these three experiments — the error/ARI numbers throughout this report should be read as "as calibrated in this repo," not as a claim about the underlying algorithms' theoretical guarantees.

10 Appendix: why Experiment 4's budget_cap increase and its memory-guard override are two independent changes

Both `baseline`'s budget_cap = 400,000 and `amplified_imbalance`'s budget_cap = 800,000 (Section 1.4) are sometimes read as "caused by" the memory-guard override. That is only half right — the two are separate, independently-necessary changes, and neither alone would have raised kabc's convergence ceiling from k≤7 to k≤8.

The `budget_cap` value itself comes from kabc's own doubling-trick cost schedule, not from the memory-guard formula. For a given N, the doubling trick's cumulative pull cost through iteration k is fixed by `thresholds.py`'s `iteration_budget`/ `iteration_confidence` (Algorithm 2), independent of any memory-guard parameter. At N=40 (`baseline`), the cumulative cost through k=8 is 360,520; at N=76 (`amplified_imbalance`), it is 735,148. Each setting's `budget_cap` was set comfortably above that number (400,000: ≈11% margin; 800,000: ≈8.8% margin) specifically so that `budget_cap` itself would no longer be the thing stopping kabc at k=7 — exactly the way the earlier design's budget_cap = 200,000 did (§1.4's superseded version): 200,000 ≪ 360,520, so k=8 was financially unaffordable under the old cap regardless of memory-guard settings.

The memory-guard override answers a categorically different question: is it even SAFE to attempt k=8's Gram-matrix computation at all. Under the original guard (11 workers, 4× safety, trip point $n_k > 2{,}852$), k=8's per-arm sample size (n_k≈4,602 at N=40; n_k≈4,933 at N=76) would have been refused outright (`_MemoryGuardExceeded`) regardless of how large `budget_cap` was set. The real-verified override (`memory_guard_n_workers=5`, `memory_guard_safety_factor=2`, both matching this experiment's actual n_{jobs}=5 and its real measured peak RSS — see Section 1.4) raised that trip point to $n_k > 5{,}982.6$, which is what makes attempting k=8 safe in the first place.

Both changes were necessary; neither alone was sufficient. Old `budget_cap` (200,000) with the new guard: still blocked at k=7 (can't afford k=8's cost). Old guard ($n_k > 2{,}852$) with the new, larger `budget_cap`: still blocked at k=7 (the guard refuses to even attempt k=8's Gram matrices, before any budget check on that iteration is reached). Only raising both together unlocked k=8 for both settings.

11 Appendix: how loose is Experiment 1's budget_cap relative to the paper's own Theorem 3.1 bound

Experiment 1's `budget_cap` (`s_star_formula`, `constant=200`: `budget_cap` = $200 \cdot N \cdot s_*^{-2} \cdot \log(N/\delta)$, `scripts/common.py`'s `_resolve_budget_cap`) is a practical, hand-chosen safety multiplier, not a value derived from the paper's own proof. This note quantifies exactly how it compares to the paper's fully-derived, proven one-shot sample-complexity bound (Theorem 3.1, Eq. 8):

$$8N\left(\frac{128}{s_*^2} \vee 1\right)\log\left(\frac{32(N^2-N)k_*^2}{\delta}\right), \qquad k_* = \left\lceil \log_2 \frac{128}{s_*^2} \right\rceil \vee 1,$$

using the *same* Monte-Carlo s_*^2 estimate (`kabc.metrics.estimate_s_star_sq`, base config, fixed seed) for both formulas, so the comparison isolates the constant/log-term difference rather than a different difficulty measure. Computed for Experiment 1's base config (N=60, K=3, d=10, σ=1, δ=0.05) across its main Δ sweep:

Δ	s_*^2 (MC est.)	`budget_cap` (ours)	k_*	Eq. 8 bound (paper)	ratio (ours / Eq. 8)
1	0.146894	579,200	10	8,046,726	0.071980
1.5	0.329325	258,350	9	3,549,893	0.072777
2	0.582866	145,971	8	1,980,894	0.073689
3	1.185555	71,765	7	960,046	0.074752
4	1.401399	60,712	7	812,179	0.074752
6	1.613719	52,724	7	705,319	0.074752

Finding. Our `budget_cap` is consistently ≈7.2–7.5% of the paper's exact Eq. 8 bound across the entire Δ sweep (i.e. Eq. 8 is ≈13.4–13.9× larger), and the ratio is remarkably stable, converging as Δ grows and k_* settles at 7.

This is a distinct knob from `c_cal` (Section 1.4's memory-guard note and the main text's calibration protocol), not the same looseness measured twice. `budget_cap` is a hard spending *ceiling* passed to `_fit` — it bounds how much `tau` an algorithm is *allowed* to spend before being forcibly stopped, and is rarely binding for kabc in practice (kabc's own calibrated stopping rule usually halts well under cap). `c_cal` (≈ 0.103516 for kabc, see `configs/calibration.yaml`) instead multiplies the per-iteration same/different-cluster decision bound (Eq. 6) inside `_cluster_once` — it governs *when kabc actually decides to stop*. The two ratios (≈0.072–0.075 for `budget_cap`/Eq. 8, ≈0.103 for `c_cal`) land in a similar order of magnitude by coincidence, not because they are the same quantity measured two ways.

References

[1] Thuot, Vogt, Ghoshdastidar, Verzelen. *Kernel-based Active Clustering of Bandit Arms*. arXiv:2601.07535, 2026.

[2] Thuot, Carpentier, Giraud, Verzelen. *Active clustering with bandit feedback*. arXiv:2406.11485, 2024.

[3] Yavas, Huang, Tan, Scarlett. *A General Framework for Clustering and Distribution Matching with Bandit Feedback*. arXiv:2409.05072, 2024.

[4] Wolfer, Alquier. *Variance-aware estimation of kernel mean embedding*. Journal of Machine Learning Research, 26(57):1–48, 2025.

[5] Tolstikhin, Sriperumbudur, Muandet. *Minimax estimation of kernel mean embeddings*. Journal of Machine Learning Research, 18(86):1–47, 2017.

[6] Yang, Zhong, Tan. *Optimal Clustering with Bandit Feedback*. Journal of Machine Learning Research, 25(186):1–54, 2024. arXiv:2202.04294.

[7] Graf, Thuot, Verzelen. *Clustering Items through Bandit Feedback: Finding the Right Feature out of Many*. Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025. arXiv:2503.11209.

[8] Yash, Karamchandani, Ghosh. *Near Optimal Best Arm Identification for Clustered Bandits*. Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025. arXiv:2505.10147.

[9] Chandran, G. Dhinesh; Reddy, Kota Srinivas; Bhashyam, Srikrishna. *Online Clustering of Data Sequences with Bandit Information*. 2025 IEEE International Symposium on Information Theory (ISIT), 2025. arXiv:2501.11421.

[10] Chandran, G. Dhinesh; Reddy, Kota Srinivas; Bhashyam, Srikrishna. *Efficient Clustering in Stochastic Bandits*. arXiv:2601.09162, 2026 (preprint).